

Appunti del corso di **Linguaggi**

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis

Matteo Drago

Indice

1	Introduzione	1
1.1	Perchè studiare i PL?	1
1.2	Cosa sono i PL?	2
1.3	Cosa significa programmare?	2
1.4	Contesti di sviluppo	3
1.5	Classificazione dei PL	3
1.6	Implementazione di un PL	4
1.7	Realizzare una macchina astratta	5
1.7.1	Soluzione interpretativa	7
1.7.1.1	Operazioni	7
1.7.1.2	Struttura	8
1.7.2	Soluzione compilativa	8
1.7.2.1	Struttura	9
1.7.3	Soluzione ibrida	9
2	Descrivere i linguaggi di programmazione	11
2.1	Sintassi	11
2.1.1	Descrivere la sintassi	12
2.1.2	Grammatiche CF	12
2.1.2.1	Notazione BNF	14
2.1.3	Descrivere un semplice linguaggio imperativo	14
2.1.3.1	Analisi semantica	15
2.1.4	Categorie sintattiche	15
2.1.4.1	Espressioni	16
2.1.4.2	Comandi	16
2.1.4.3	Dichiarazioni	16
2.1.5	Descrivere un semplice linguaggio implementato	17
2.1.5.1	Sintassi	17
2.2	Semantica	18
2.2.1	Induzione matematica	18
2.2.2	Induzione strutturale	19
2.2.3	Proprietà della semantica	19
2.2.4	Tipi di semantica	19
2.2.4.1	Semantica denotazionale	20
2.2.4.2	Semantica assiomatica	21
2.2.4.3	Semantica operativa	22

3	Espressioni	24
3.1	Dscrivere le espressioni	24
3.1.1	Notazione	24
3.1.1.1	Notazione in-fissa	25
3.1.1.2	Notazione pre-fissa	25
3.1.1.3	Notazione post-fissa	26
3.1.2	Regole di precedenza	26
3.1.3	Regole di associatività	26
3.1.4	Valutazione delle espressioni	27
3.2	Sintassi	27
3.3	Semantica	28
3.3.1	Regole di transizione: espressioni aritmetiche	28
3.3.1.1	Regole di transizione: espressioni booleane	29
4	Identificatori	31
4.1	Legame	32
4.1.1	Tipi di binding	33
4.1.2	Tempo di binding	33
4.2	Ambiente	33
4.3	Sintassi	33
4.3.1	Identificatore libero	34
4.4	Semantica	35
4.4.1	Regole di transizione	35
4.4.2	Tipo di dati	36
4.4.2.1	Legami di tipo	37
4.4.3	Semantica statica	37
4.4.3.1	Ambiente statico	37
4.4.3.2	Semantica statica delle espressioni	37
4.4.3.3	Compatibilità di ambienti	39
5	Dichiarazioni	40
5.1	Sintassi	40
5.1.1	Modifica dell'ambiente	42
5.1.2	Identificatori definiti	42
5.1.3	Identificatori liberi	43
5.2	Semantica	43
5.2.1	Semantica statica	43
5.2.2	Semantica dinamica	44
6	Variabili	47
7	Comandi	49
7.1	Sintassi	49
7.2	Semantica	49
7.2.1	Semantica statica	49
7.2.2	Memoria	50
7.2.3	Aggiornamento della memoria	51
7.2.4	Semantica dinamica	51
7.3	Strutture di controllo	53

7.3.1	Comandi di selezione	53
7.3.1.1	Selettori a due vie	53
7.3.1.1.1	Semantica statica	54
7.3.1.1.2	Semantica dinamica	54
7.3.1.2	Selettori a più vie	54
8	Procedure	55

Procedure

L'uso delle procedure permette l'**astrazione del controllo**.

Nota

Astrarre significa nascondere dettagli facendo emergere caratteristiche comuni.

L'astrazione chiede di identificare le proposizioni importanti che si vuole descrivere al fine di concentrarsi che si vuole descrivere al fine di concentrarsi sulle questioni rilevanti e ignorare quelle non importanti.

Quando si parla di astrazione del controllo ci sono, in realtà, due categorie di sottoprogrammi con nome che possiamo definire:

- ◇ **procedure** (no output)
- ◇ **funzioni** (restituisce output)

Parte importante del processo di astrazione consiste nel dare al raggruppamento un nome.

Il **nome** permette di riferirsi all'astrazione senza conoscerne i dettagli, una sola le caratteristiche di interesse che ne hanno determinato il raggruppamento.

I **parametri** permettono l'astrazione di tante computazioni simili che cambiano di tante computazioni simili che cambiano solo per alcuni input a cui vengono applicate e che vengono abbreviate con un singolo nome.

Il **corpo** è ciò che viene valutato/eleborato/eseguito ogni volta che il nome viene richiamato.

Una procedura viene definita in tre fasi:

1. **specifica**: descrizione dell'interfaccia della procedura/funzione

```
1  double P(int x) {
2
3  }
4
```

2. **definizione**: descrizione del corpo della procedura/funzione

```
1  {
2      double y;
3      //corpo procedura P
```

```
4      ...  
5      (return y);  
6  }  
7
```

3. **uso**: chiamata della procedura/funzione per eseguire il codice

```
1      int n;  
2      double z;  
3      z = P(n); //chiamata della procedura  
4
```

Consideriamo il seguente codice:

```
1      int foo(int n) {  
2          int tmp = a;  
3  
4          if(tmp == 0)  
5              return n;  
6          else  
7              return n + 1;  
8      }
```

Listing 8.1: programma di esempio

In questo caso la variabile `a` è un identificatore non locale, cioè un identificatore libero per cui di deve trovare il significato nel contesto.